Object Oriented Programming

Prof. Dr. Muhammad Usman
Department of Computer Science
UET Mardan

Email: usman@uetmardan.edu.pk

MODULARIZED APPROACH

Modules: *functions* and *classes*

Programs use new and "prepackaged" modules

- New: programmer-defined functions, classes
- Prepackaged: from the standard library

A named block of code that may receive values and may return a value is called a function

Functions invoked by function call

- Function name and information (arguments) it needs

Function definitions

- Only written once
- Hidden from other functions

MATH LIBRARY FUNCTIONS

Perform common mathematical calculations

- Include the header file **<cmath>**

Functions called by writing

- functionName (argument);

or

- functionName(argument1, argument2, ...);

Example

```
cout << sqrt( 900.0 );
```

- sqrt (square root) function The preceding statement would print 30
- All functions in math library return a **double**

MATH LIBRARY FUNCTIONS

Function arguments can be

- Constants
  - `sqrt( 4 );`
- Variables
  - `sqrt( x );`
- Expressions
  - `sqrt( sqrt( x ) ) ;`
  - `sqrt( 3 - 6x );`

Other functions

- `ceil(x), floor(x), log10(x), etc.`

MODULARIZED APPROACH

Functions

- Modularize a program
- Software reusability
  - Call function multiple times

Local variables

- Known only in the function in which they are defined
- All variables declared in function definitions are local variables

Parameters

- Local variables passed to function when called
- Provide outside information

MODULARIZED APPROACH

Function prototype

- Tells compiler argument type and return type of function
  - Function takes an **`int`** and returns an **`int`**

```
int square( int );
```

Calling/invoking a function

- Write function name with argument as variable, expression or value.

```
square( x );
```

Format for function definition

return-value-type *function-name* **(** *parameter-list* **)**
{
  declarations and statements
}

```
int square( int x ){
  return x*x;
}
```

Prototype must match function definition

- Function prototype

```
double maximum( double, double, double );
```

- Definition

```
double maximum( double x, double y, double z )
{
  ...
}
```

MODULARIZED APPROACH

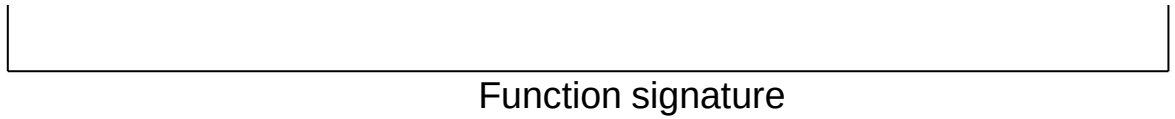
`return` keyword

- Returns data, and control goes to function's caller
  - If no data to return, use `return;`
- Function ends when reaches right brace
  - Control goes to caller

Functions cannot be defined inside other functions

Function signature

- Part of prototype with name and parameters
  - `double maximum( double, double, double );`



MODULARIZED APPROACH

Argument Coercion. - Type Casting

- Force arguments to be of proper type
  - Converting **`int`** (4) to **`double`** (4.0)
- Conversion rules
  - Arguments usually converted automatically
  - Changing from **`double`** to **`int`** can truncate data
    - 3.4 to 3
  - Mixed type goes to highest type (promotion)
    - **`int * double`**

```
cout << sqrt(4)
```

```
int x = int(45.0);
```

```
int x = (int)45.0;
```

Type Conversion / Casting

1. Coercion / Implicit type Conversion / promotion
2. Truncation / Explicit type Conversion

VARIABLE TYPES

TABLE 2.2 Basic C++ Variable Types

Keyword	*Numerical Range* *Low*	*High*	*Digits of Precision*	*Bytes of Memory*
`bool`	false	true	n/a	1
`char`	–128	127	n/a	1
`short`	–32,768	32,767	n/a	2
`int`	–2,147,483,648	2,147,483,647	n/a	4
`long`	–2,147,483,648	2,147,483,647	n/a	4
`float`	3.4×10^{-38}	3.4×10^{38}	7	4
`double`	1.7×10^{-308}	1.7×10^{308}	15	8

TABLE 2.3 Unsigned Integer Types

Keyword	*Numerical Range*		*Bytes of Memory*
	Low	*High*	
`unsigned char`	0	255	1
`unsigned short`	0	65,535	2
`unsigned int`	0	4,294,967,295	4
`unsigned long`	0	4,294,967,295	4

AUTOMATIC CONVERSION

TABLE 2.4 Order of Data Types

Data Type	*Order*
`long double`	Highest
`double`	
`float`	
`long`	
`int`	
`short`	
`char`	Lowest

HEADER FILES

Header files contain

- Function prototypes
- Definitions of data types and constants

Header files ending with .h

- Programmer-defined header files

```
#include "myheader.h"
```

Library header files

```
#include <cmath>
```

STORAGE CLASSES

Variables have attributes

- Have seen name, type, size, value
- Storage class
  - How long variable exists in memory
- Scope
  - Where variable can be referenced in program
- Linkage
  - For multiple-file program, which files can use it

STORAGE CLASSES

Automatic storage class

- Variable created when program enters its block
- Variable destroyed when program leaves block
- Only local variables of functions can be automatic
  - Automatic by default
  - keyword **`auto`** explicitly declares automatic
- **`register`** keyword
  - Hint to place variable in high-speed register
  - Good for often-used items (loop counters)
  - Often unnecessary, compiler optimizes
- Specify either **`register`** or **`auto`**, not both
  - `register int counter = 1;`

STORAGE CLASSES

Static storage class

- Variables exist for entire program
  - For functions, name exists for entire program
- May not be accessible, scope rules still apply

`auto` and `register` keyword

- Created and active in a block
- local variables in function
- `register` variables are kept in CPU registers

`static` keyword

- Local variables in function
- Keeps value between function calls
- Only known in own function

`extern` keyword

- Default for global variables/functions
  - Globals: defined outside of a function block
- Known in any function that comes after it

SCOPE RULES

Scope

- Portion of program where identifier can be used

File scope

- Defined outside a function, known in all functions
- Global variables, function definitions and prototypes

Function scope

- Can only be referenced inside defining function
- Only labels, e.g., identifiers with a colon (`case:`)

SCOPE RULES

Block scope

- Begins at declaration, ends at right brace `}`
  - Can only be referenced in this range
- Local variables, function parameters
- **`static`** variables still have block scope
  - Storage class separate from scope

Function-prototype scope

- Parameter list of prototype
- Names in prototype optional
  - Compiler ignores
- In a single prototype, name can be used once

```
3   #include <iostream>
4
5   using std::cout;
6   using std::endl;
7
8   void useLocal( void );        // function prototype
9   void useStaticLocal( void );  // func
10  void useGlobal( void );       // func
11
12  int x = 1;      // global variable
13
14  int main()
15  {
16    int x = 5;   // local variable to main
17
18    cout << "local x in main's outer scope
19
20    { // start new scope
21
22      int x = 7;
23
24      cout << "local x in main's inner scope is " << x << endl;
25
26    } // end new scope
```

Declared outside of function; global variable with file scope.

Local variable with function scope.

Create a new block, giving **x** block scope. When the block ends, this **x** is destroyed.

```
27
28      cout << "local x in main's outer scope is " << x << endl;
29
30      useLocal();       // useLocal has local x
31      useStaticLocal(); // useStaticLocal has static local x
32      useGlobal();      // useGlobal uses global x
33      useLocal();       // useLocal reinitializes its local x
34      useStaticLocal(); // static local x retains its prior value
35      useGlobal();      // global x also retains its value
36
37      cout << "\nlocal x in main is " << x << endl;
38
39      return 0;   // indicates successful termination
40
41  } // end main
42
```

```
43  // useLocal reinitializes local variable x during each call
44  void useLocal( void )
45  {
46     int x = 25;  // initialized each time useLocal is called
47
48     cout << endl << "local
49          << " on entering use
50     ++x;
51     cout << "local x is " << x
52          << " on exiting useLocal" << endl;
53
54  } // end function useLocal
55
```

Automatic variable (local variable of function). This is destroyed when the function exits, and reinitialized when the function begins.

```
56 // useStaticLocal initializes static local variable x only the
57 // first time the function is called; value of x is saved
58 // between calls to this function
59 void useStaticLocal( void )
60 {
61    // initialized only first time useStaticLocal is called
62    static int x = 50;
63
64    cout << endl << "local static x is " <<
65         << " on entering useStaticLocal"
66    ++x;
67    cout << "local static x is " << x
68         << " on exiting useStaticLocal" << endl;
69
70 } // end function useStaticLocal
```

Static local variable of function; it is initialized only once, and retains its value between function calls.

```
72  // useGlobal modifies global variable x during each call
73  void useGlobal( void )
74  {
75     cout << endl << "global x is " << x
76          << " on entering useGlobal" << endl;
77     x *= 10;
78     cout << "global x is " << x
79          << " on exiting useGlobal" << endl;
80
81  } // end function useGlobal
```

This function does not declare any variables. It uses the global **x** declared in the beginning of the program.

```
local x in main's outer scope is 5
local x in main's inner scope is 7
local x in main's outer scope is 5

local x is 25 on entering useLocal
local x is 26 on exiting useLocal

local static x is 50 on entering useStaticLocal
local static x is 51 on exiting useStaticLocal

global x is 1 on entering useGlobal
global x is 10 on exiting useGlobal
```

RECURSION

Calculating the value of a function on larger input in terms of the value of the same function on smaller input

Recursive functions

- Functions that call themselves
- Can only solve a base case

If not base case

- Break problem into smaller problem(s)
- Launch new copy of function to work on the smaller problem (recursive call/recursive step)
  - Slowly converges towards base case
  - Function makes call to itself inside the return statement
- Eventually base case gets solved
  - Answer works way back up, solves entire problem

RECURSION

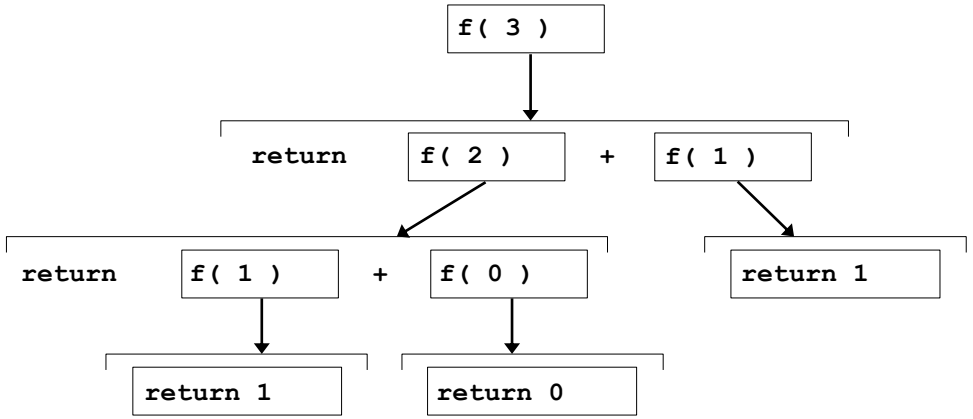
Factorial: n! = n * (n – 1) * (n – 2) * … * 1

- Recursive relationship (n! = n * (n – 1)!)

  *5! = 5 * 4!*

  *4! = 4 * 3!…*

- Base case (1! = 0! = 1)



Fibonacci series: 0, 1, 1, 2, 3, 5, 8…

- Each number sum of two previous ones
- Example of a recursive formula:
  - fib(n) = fib(n-1) + fib(n-2)

```
long fibonacci( long n ){
   if ( n == 0 || n == 1 )  // base case
        return n;
  else
   return fibonacci( n - 1 ) + fibonacci( n – 2 );
}
```

```
1   // Recursive factorial function.
3   #include <iostream>
4
5   using std::cout;
6   using std::endl;
7
8   #include <iomanip>
9
10  using std::setw;
11
12  unsigned long factorial( unsigned long ); // function prototype
13
14  int main()
15  {
16     // Loop 10 times. During each iteration, calculate
17     // factorial( i ) and display result.
18     for ( int i = 0; i <= 10; i++ )
19        cout << setw( 2 ) << i << "! = "
20             << factorial( i ) << endl;
21
22     return 0;  // indicates successful termination
23
24  } // end main
```

Data type **`unsigned long`** can hold an integer from 0 to 4 billion.

```
26  // recursive definition of function factorial
27  unsigned long factorial( unsigned long number )
28  {
29     // base case
30     if ( number <= 1 )
31        return 1;
32
33     // recursive step
34     else
35        return number * factorial( number - 1 );
36
37  } // end function factorial
```

The base case occurs when we have `0!` or `1!`. All other cases must be split up (recursive step).

```
 0! = 1
 1! = 1
 2! = 2
 3! = 6
 4! = 24
 5! = 120
 6! = 720
 7! = 5040
 8! = 40320
 9! = 362880
10! = 3628800
```

EXAMPLE USING RECURSION: FIBONACCI SERIES

Fibonacci series: 0, 1, 1, 2, 3, 5, 8...

- Each number sum of two previous ones
- Example of a recursive formula:
  - *fib(n) = fib(n-1) + fib(n-2)*

C++ code for Fibonacci function

```
long fibonacci( long n )
{
 if ( n == 0 || n == 1 )  // base case
     return n;
  else
   return fibonacci( n - 1 ) +
          fibonacci( n - 2 );
}
```

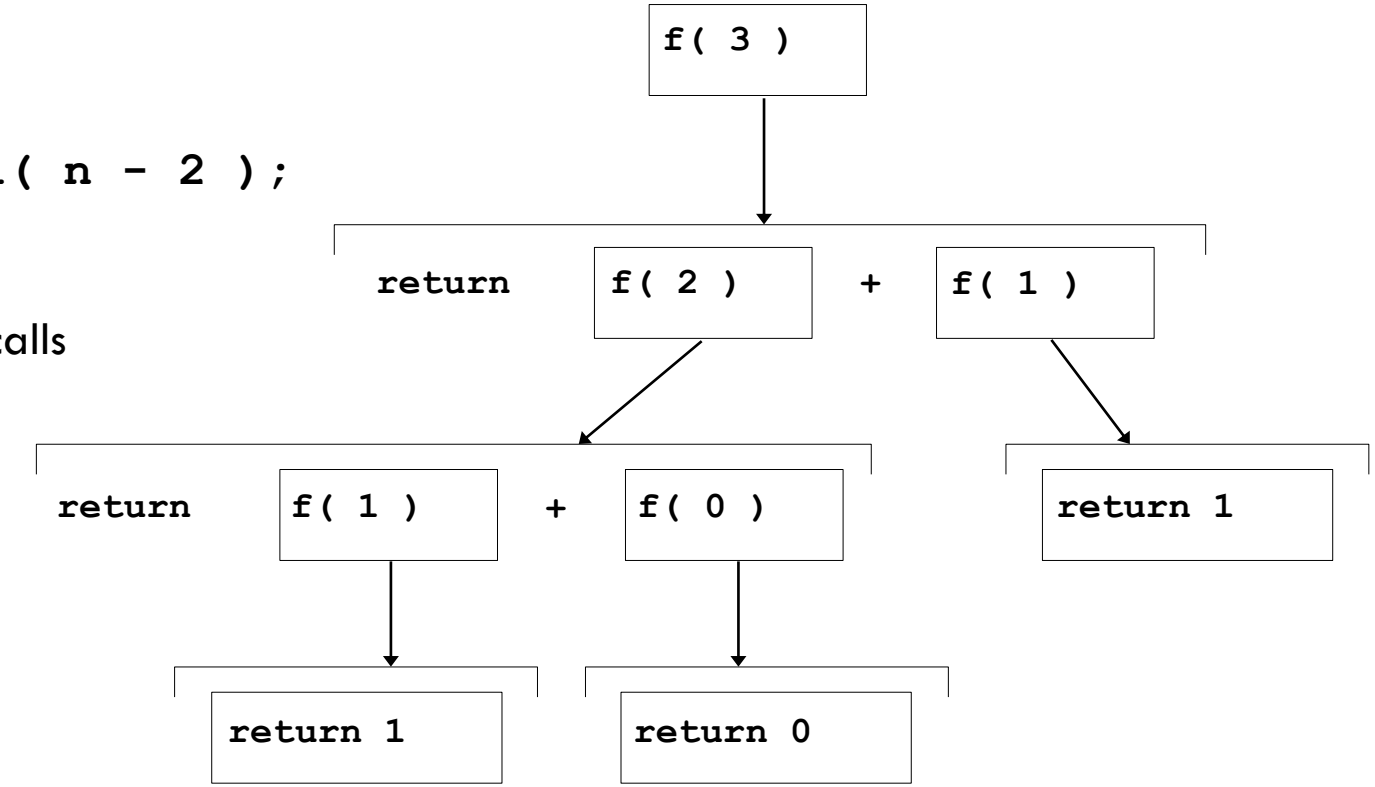
EXAMPLE USING RECURSION: FIBONACCI SERIES

Order of operations

- `return fibonacci( n - 1 ) + fibonacci( n - 2 );`

Recursive function calls

- Each level of recursion doubles the number of function calls
  - 30th number = 2^30 ~ 4 billion function calls
- Exponential complexity



RECURSION VS. ITERATION

Repetition

- Iteration: explicit loop
- Recursion: repeated function calls

Termination

- Iteration: loop condition fails
- Recursion: base case recognized

Both can have infinite loops

Balance between performance (iteration) and good software engineering (recursion)

INLINE FUNCTIONS

- Keyword `inline` before function (at the place of function definition)
- Asks the compiler to copy code into program instead of making function call
  - Reduce function-call overhead
  - Request, Not Command (ignored if the function is too large, complex or recursive)
  - Compiler can ignore `inline`
- Good for small, often-used functions
- Example

```
inline double cube(const double s ){
return s * s * s;
}
```

`//const` tells compiler that function does not modify `s`

```
2   // Using an inline function to calculate.
3   // the volume of a cube.
4   #include <iostream>
5
6   using std::cout;
7   using std::cin;
8   using std::endl;
9
10  // Definition of inline function cube. Definition of function
11  // appears before function is called, so a function prototype
12  // is not required. First line of function definition acts as
13  // the prototype.
14  inline double cube( const double side )
15  {
16     return side * side * side;  // calculate cube
17
18  } // end function cube
```

```
20  int main()
21  {
22     cout << "Enter the side length of your cube: ";
23
24     double sideValue;
25
26     cin >> sideValue;
27
28     // calculate cube of sideValue and display result
29     cout << "Volume of cube with side "
30          << sideValue << " is " << cube( sideValue ) << endl;
31
32     return 0;  // indicates successful termination
33
34  } // end main
```

```
Enter the side length of your cube: 3.5
Volume of cube with side 3.5 is 42.875
```

REFERENCES AND REFERENCE PARAMETERS

Call by value

- Copy of data passed to function
- Changes to copy do not change original
- Prevent unwanted side effects

Call by reference

- Function can directly access data
- Changes affect original

Reference parameter

- Alias for argument in function call
  - Passes parameter by reference
- Use **&** after data type in prototype
  - **`void myFunction( int &data )`**
  - Read "**`data`** is a reference to an **`int`**"
- Function call format the same
  - However, original can now be changed

```
2   // Comparing pass-by-value and pass-by-reference
3   // with references.
4   #include <iostream>
5
6   using std::cout;
7   using std::endl;
8
9   int squareByValue( int );        // function prototype
10  void squareByReference( int & );  // function prototype
11
12  int main()
13  {
14     int x = 2;
15     int z = 4;
16
17     // demonstrate squareByValue
18     cout << "x = " << x << " before squareByValue\n";
19     cout << "Value returned by squareByValue: "
20          << squareByValue( x ) << endl;
21     cout << "x = " << x << " after squareByValue\n" << endl;
```

Notice the **&** operator, indicating pass-by-reference.

```
23    // demonstrate squareByReference
24    cout << "z = " << z << " before squareByReference" << endl;
25    squareByReference( z );
26    cout << "z = " << z << " after squareByReference" << endl;
27
28    return 0;  // indicates successful termination
29 } // end main
30
31 // squareByValue multiplies number by itself, stores the
32 // result in number and returns the new value of numbe
33 int squareByValue( int number )
34 {
35    return number *= number;  // caller's argument not modified
36
37 } // end function squareByValue
38
39 // squareByReference multiplies numberRef by itself and
40 // stores the result in the variable to which numberRef
41 // refers in function main
42 void squareByReference( int &numberRef )
43 {
44    numberRef *= numberRef;   // caller's argument modified
45
46 } // end function squareByReference
```

Changes **`number`**, but original parameter (**`x`**) is not modified.

Changes **`numberRef`**, an alias for the original parameter. Thus, **`z`** is changed.

```
x = 2 before squareByValue
Value returned by squareByValue: 4
x = 2 after squareByValue


z = 4 before squareByReference
z = 16 after squareByReference
```

REFERENCES AND REFERENCE PARAMETERS

Pointers

- Another way to pass-by-refernce

References as aliases to other variables

- Refer to same variable
- Can be used within a function

```
int count = 1;       // declare integer variable count
Int &cRef = count; // create cRef as an alias for count
++cRef; // increment count (using its alias)
```

References must be initialized when declared

- Otherwise, compiler error
- Dangling reference
  - Reference to undefined variable

```
2   // References must be initialized.
3   #include <iostream>
4
5   using std::cout;
6   using std::endl;
7
8   int main()
9   {
10     int x = 3;
11
12     // y refers to (is an alias for) x
13     int &y = x;
14
15     cout << "x = " << x << endl << "y = " << y << endl;
16     y = 7;
17     cout << "x = " << x << endl << "y = " << y << endl;
18
19     return 0;  // indicates successful termination
20
21  } // end main
```


```
x = 3
y = 3
x = 7
y = 7
```

```
2   // References must be initialized.
3   #include <iostream>
4
5   using std::cout;
6   using std::endl;
7
8   int main()
9   {
10     int x = 3;
11     int &y;     // Error: y must be initialized
12
13     cout << "x = " << x << endl << "y = " << y << endl;
14     y = 7;
15     cout << "x = " << x << endl << "y = " << y << endl;
16
17     return 0;  // indicates successful termination
18
19  } // end main
```

Uninitialized reference – compiler error.

```
Borland C++ command-line compiler error message:
 Error E2304 Fig03_22.cpp 11: Reference variable 'y' must be
   initialized in function main()

Microsoft Visual C++ compiler error message:
 D:\cpphtp4_examples\ch03\Fig03_22.cpp(11) : error C2530: 'y' :
   references must be initialized
```

DEFAULT ARGUMENTS

Function call with omitted parameters

- If not enough parameters, rightmost go to their defaults
- Default values
  - Can be constants, global variables, or function calls

Set defaults in function prototype

```
int myFunction( int x = 1, int y = 2, int z = 3 );
```

```
myFunction( 3 );
```

`x = 3`, `y` and `z` get defaults (rightmost)

```
myFunction( 3, 5 );
```

`x = 3`, `y = 5` and `z` gets default

UNITARY SCOPE RESOLUTION OPERATOR

Unary scope resolution operator (: :)

- Access global variable if local variable has same name
- Not needed if names are different
- Use **`::variable`**
  - **`y = ::x + 3;`**
- Good to avoid using same names for locals and globals

```
int x = 10; // Global x
int main() {
    int x = 5; // Local x
    std::cout << "Local: " << x << std::endl;
    std::cout << "Global: " << ::x << std::endl; //
Accessing global x
}
```

FUNCTION OVERLOADING

- Functions with same name and different parameters
  - Should perform similar tasks
  - i.e., function to square `int`s and function to square `float`s
- Overloaded functions distinguished by signature
  - Based on name and parameter types (order matters)
  - Name mangling
    - Encodes function identifier with parameters
  - Type-safe linkage
    - Ensures proper overloaded function called

```
int square( int x){
return x * x;
}

float square(float x){
 return x * x;
}
```

FUNCTION TEMPLATES

- To write a single, generic function that works with different data types.
- Instead of writing multiple overloaded functions for int, double, or float, create a "blueprint" that the compiler uses to generate the specific code needed at compile time.

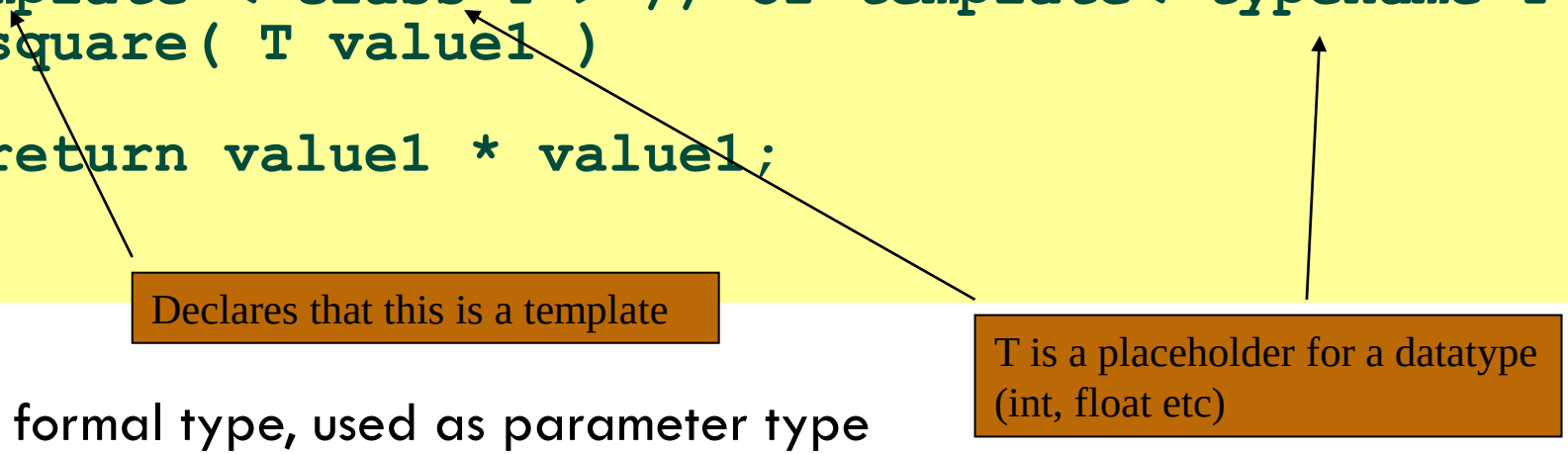
- Compact way to make overloaded functions
  - Generate separate function for different data types
- Format
  - Begin with keyword **`template`**
  - Formal type parameters in brackets **`<>`**
    - Every type parameter preceded by **`typename`** or **`class`** (synonyms)
    - Placeholders for built-in types (i.e., **`int`**) or user-defined types
    - Specify arguments types, return types, declare variables
  - Function definition like normal, except formal types used

```
template < class T >
// or template< typename T >

T square( T value1 )
{
   return value1 * value1;
}
```

FUNCTION TEMPLATES

```
template < class T > // or template< typename T >
T square( T value1 )
{
   return value1 * value1;
}
```



- **T** is a formal type, used as parameter type
  - Above function returns variable of same type as parameter
- In function call, T replaced by real type
  - If **int**, all **T**'s become **int**s

```
int x;
int y = square(x);
```

Call template function

Explicit Specification: cout << square<double>(2.5, 3.5);

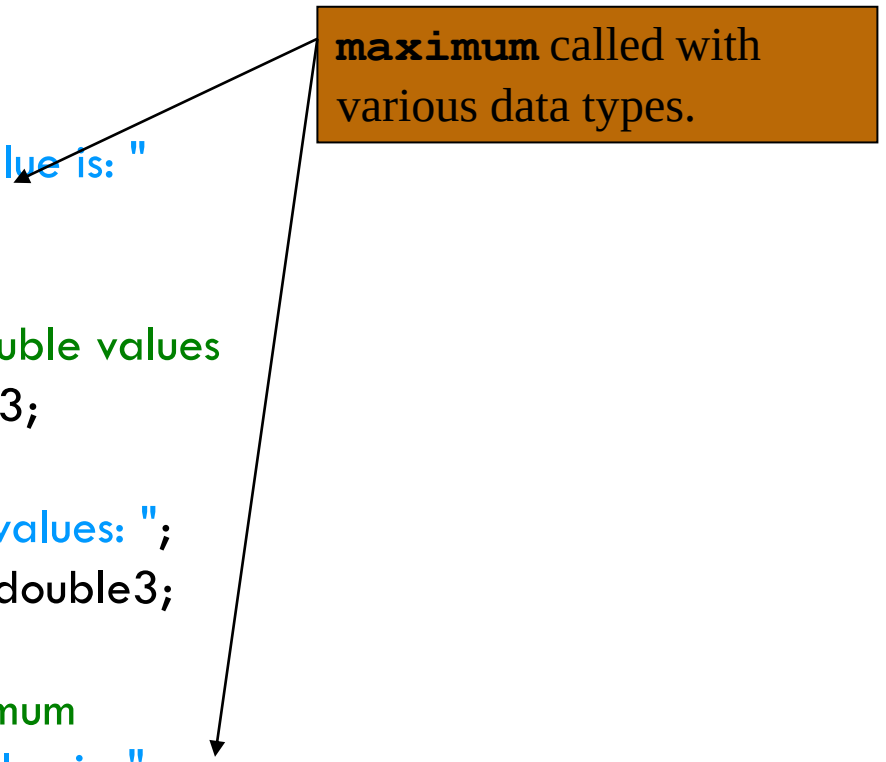
Template Argument Deduction: cout << square(2.5, 3.5);

```
2   // Using a function template.
3   #include <iostream>
4
5   using std::cout;
6   using std::cin;
7   using std::endl;
8
9   // definition of function template
10  template < class T >  // or template < typename T >
11  T maximum( T value1, T value2, T value3 )
12  {
13     T max = value1;
14
15     if ( value2 > max )
16        max = value2;
17
18     if ( value3 > max )
19        max = value3;
20
21     return max;
22
23  } // end function template maximum
24
```

Formal type parameter **`T`** placeholder for type of data to be tested by **`maximum`**.

`maximum` expects all parameters to be of the same type.

```
25  int main()
26  {
27     // demonstrate maximum with int values
28     int int1, int2, int3;
29
30     cout << "Input three integer values: ";
31     cin >> int1 >> int2 >> int3;
32
33     // invoke int version of maximum
34     cout << "The maximum integer value is: "
35          << maximum( int1, int2, int3 );
36
37     // demonstrate maximum with double values
38     double double1, double2, double3;
39
40     cout << "\n\nInput three double values: ";
41     cin >> double1 >> double2 >> double3;
42
43     // invoke double version of maximum
44     cout << "The maximum double value is: "
45          << maximum( double1, double2, double3 );
46
```



```
47     // demonstrate maximum with char values
48     char char1, char2, char3;
49
50     cout << "\n\nInput three characters: ";
51     cin >> char1 >> char2 >> char3;
52
53     // invoke char version of maximum
54     cout << "The maximum character value is: "
55          << maximum( char1, char2, char3 )
56          << endl;
57
58     return 0;  // indicates successful termination
59
60  } // end main
```

```
Input three integer values: 1 2 3
The maximum integer value is: 3

Input three double values: 3.3 2.2 1.1
The maximum double value is: 3.3

Input three characters: A C B
The maximum character value is: C
```

ARRAYS

Array

- A collection of same datatype data, having a common name, a unique index and are store in consecutive memory locations

To refer to an element

- Specify array name and position number (index)
- Format: arrayname[position number]
- First element at position 0

N-element array c

```
c[ 0 ], c[ 1 ] ... c[ n - 1 ]
```

- Nth element as position N-1

ARRAYS

Array elements are treated like other variables

- Assignment, printing for an integer array `c`

```
c[ 0 ] =  3;
cout << c[ 0 ];
```

Can perform operations inside subscript

`c[ 5 - 2 ]` same as `c[3]`

DECLARING ARRAYS

When declaring arrays, specify

- Name
- Type of array
  - Any data type
- Number of elements
- *type arrayName* [*arraySize*] ;

```
int c[ 10 ];  // array of 10 integers
float d[ 3284 ]; // array of 3284 floats
```

Declaring multiple arrays of same type

- Use comma separated list, like regular variables

```
int b[ 100 ], x[ 27 ];
```

INITIALIZING ARRAYS

- For loop
  - Set each element
- Initializer list
  - Specify each element when array declared

```
int n[ 5 ] = { 1, 2, 3, 4, 5 };
```

  - If not enough initializers, rightmost elements 0
  - If too many syntax error
- To set every element to 0 value

```
int n[ 5 ] = { 0 };
```

- If array size omitted, initializers determine size

```
int n[] = { 1, 2, 3, 4, 5 };
```

  - 5 initializers, therefore 5 element array

EXAMPLES USING ARRAYS

Strings

- Arrays of characters
- All strings end with **`null`** (**`'\0'`**)
- Examples
  - **`char string1[] = "hello";`**
    - **`Null`** character implicitly added
    - **`string1`** has 6 elements
  - **`char string1[] = { 'h', 'e', 'l', 'l',  'o', '\0' };`**
- Subscripting is the same

  `String1[ 0 ]` is **`'h'`**

  `string1[ 2 ]` is **`'l'`**

EXAMPLES USING ARRAYS

Input from keyboard

```
char string2[ 10 ];
cin >> string2;
```

- Puts user input in string
  - Stops at first whitespace character
  - Adds **`null`** character
- If too much text entered, data written beyond array
  - We want to avoid this

Printing strings

- **`cout << string2 << endl;`**
  - Does not work for other array types
- Characters printed until **`null`** found

EXAMPLES USING ARRAYS

Recall static storage

- If `static`, local variables save values between function calls
- Visible only in function body
- Can declare local arrays to be static
  - Initialized to zero

```
static int array[3];
```

If not static

- Created (and destroyed) in every function call

PASSING ARRAYS TO FUNCTIONS

Specify name without brackets

- To pass array **`myArray`** to **`myFunction`**
- Array size usually passed, but not required
  - Useful to iterate over all elements

```
int myArray[ 24 ];
myFunction( myArray, 24 );
```

Arrays passed-by-reference

- Functions can modify original array data
- Name of array is address of first element
  - Function knows where the array is stored
  - Can change original memory locations

Individual array elements passed-by-value

- Like regular variables

```
square( myArray[3] );
```

PASSING ARRAYS TO FUNCTIONS

Functions taking arrays

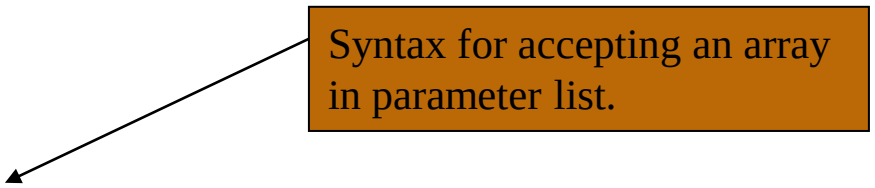
- Function prototype

```
void modifyArray( int b[], int arraySize );
void modifyArray( int [], int );
```

  - Names optional in prototype
  - Both take an integer array and a single integer
- No need for array size between brackets
  - Ignored by compiler
- If declare array parameter as **`const`**
  - Cannot be modified (compiler error)

```
void doNotModify( const int [] );
```

```
2       // Passing arrays and individual array elements to functions.
3       #include <iostream>
4
5       using std::cout;
6       using std::endl;
7
8       #include <iomanip>
9
10    using std::setw;
11
12    void modifyArray( int [], int );  // appears strange
13    void modifyElement( int );
14
15    int main()
16    {
17       const int arraySize = 5;                  // size of array a
18       int a[ arraySize ] = { 0, 1, 2, 3, 4 };   // initialize a
19
20       cout << "Effects of passing entire array by reference:"
21            << "\n\nThe values of the original array are:\n";
22
23       // output original array
24       for ( int i = 0; i < arraySize; i++ )
25          cout << setw( 3 ) << a[ i ];
```



```
27       cout << endl;
28
29       // pass array a to modifyArra
30       modifyArray( a, arraySize );
31
32       cout << "The values of the modified array are:\n";
33
34       // output modified array
35       for ( int j = 0; j < arraySize; j++ )
36          cout << setw( 3 ) << a[ j ];
37
38       // output value of a[ 3 ]
39       cout << "\n\n\n"
40            << "Effects of passing array element by value:"
41            << "\n\nThe value of a[3] i
42
43       // pass array element a[ 3 ] by
44       modifyElement( a[ 3 ] );
45
46       // output value of a[ 3 ]
47       cout << "The value of a[3] is " << a[ 3 ] << endl;
48
49       return 0;  // indicates successful termination
50
51    } // end main
```

Pass array name (**a**) and size to function. Arrays are passed-by-reference.

Pass a single array element by value; the original cannot be modified.

```
53    // in function modifyArray, "b" points to
54    // the original array "a" in memory
55    void modifyArray( int b[], int sizeOfArray )
56    {
57       // multiply each array element by 2
58       for ( int k = 0; k < sizeOfAr
59          b[ k ] *= 2;
60
61    } // end function modifyArray
62
63    // in function modifyElement, "e" is a local copy
64    // array element a[ 3 ] passed from main
65    void modifyElement( int e )
66    {
67       // multiply parameter by 2
68       cout << "Value in modifyElement is "
69            << ( e *= 2 ) << endl;
70
71    } // end function modifyElement
```

Although named **b**, the array points to the original array **a**. It can modify **a**'s data.

Individual array elements are passed by value, and the originals cannot be changed.

```
Effects of passing entire array by
reference:

The values of the original array are:

  0   1   2   3   4

The values of the modified array are:

  0   2   4   6   8

Effects of passing array element by
value:

The value of a[3] is 6

Value in modifyElement is 12

The value of a[3] is 6
```

```
2  // Demonstrating the const type qualifier.
3  #include <iostream>
4
5  using std::cout;
6  using std::endl;
7
8  void tryToModifyArray( const int [] );  // function prototype
9
10 int main()
11 {
12    int a[] = { 10, 20, 30 };
13
14    tryToModifyArray( a );
15
16    cout << a[ 0 ] << ' ' << a[ 1 ] << ' ' << a[ 2 ] << '\n';
17
18    return 0;  // indicates successful termination
19
20 } // end main
```

Array parameter declared as **`const`**. Array cannot be modified, even though it is passed by reference.

```
22  // In function tryToModifyArray, "b" cannot be used
23  // to modify the original array "a" in main.
24  void tryToModifyArray( const int b[] )
25  {
26     b[ 0 ] /= 2;    // error
27     b[ 1 ] /= 2;    // error
28     b[ 2 ] /= 2;    // error
29
30  } // end function tryToModifyArray
```

```
d:\cpphtp4_examples\ch04\Fig04_15.cpp(26) : error C2166:
   l-value specifies const object
d:\cpphtp4_examples\ch04\Fig04_15.cpp(27) : error C2166:
   l-value specifies const object
d:\cpphtp4_examples\ch04\Fig04_15.cpp(28) : error C2166:
   l-value specifies const object
```